Syllabus Highlights

COP 3520C – Computer Science I

Spring 2026

Yancy Vance Paredes, PhD

Lecturer

Yancy Vance Paredes, PhD

HEC 430A

See *Instructional Team Information* on Webcourses for Office Hours

Course Description

- A course that covers elementary **data structures**, **algorithm analysis**, and **problem-solving techniques**

- If you are a CS Major, it is a prerequisite for the department's **Foundation Exam (FE)**

Topics Covered in COP 3223

- C Syntax and Data Types
- Executable Statements
- Input/Output
- Arithmetic Expressions
- Built-in C Functions
- Control Structures
- Relational Expressions
- Logical Expressions
- Conditional Statements
- Loops
- User-Defined Functions
- Arrays (1D and 2D)
- **Pointers**
- Files
- Strings
- **Structures**
- **Dynamic Memory Allocation**
- Problem-Solving Strategies

Insertion and **lookup** will be recurring tasks throughout the semester

Insert the number 40 in the *list*

30	20	10	50	

Does the *list* contain the number 10?

`30`	`20`	`10`	`50`	`40`

Can I **correctly** solve this problem?

Is this solution or approach ***appropriate*** for the task at hand?

How can I tell if my approach is appropriate?

Topics Covered in COP 3502

- C Review
- **Dynamic Memory Allocation**
- **Recursion**
- Linked Lists
- Stacks
- Queues
- **Algorithm Analysis**
- **Summations**
- **Recurrence Relations**
- Sorting Algorithms
- Binary Search Trees
- AVL Trees
- Heaps
- Tries
- Hash Tables
- Base Conversion
- Bitwise Operators

Class Schedule

- Two 75-minute lectures (TR) – led by the lecturer
  - 6:00-7:15 PM
  - NSC 101
- One 50-minute lab – led by the TA
  - Practice Programming Problems (8-10)
  - Guided Problem Solving

Recommended Reading

- **Data Structures, Algorithms and Software Principles in C** by Thomas Standish

- **Data Structures and Algorithms in Java** by Adam Drozdek

- **Introduction to Algorithms** by Cormen, T. H., Leiserson, C. E., and Rivest, R. L., C. Stein, The MIT Press, 4th edition

These texts are optional and supplemental, yet I strongly recommend them as they will greatly support your understanding.

Grading /1

*Tentative

Assessment	Weight
Midterm Exam (March 8)	20%
Final Exam (May 2)	25%
Individual Programming Assignments (6)	25%
Quizzes (4-5)	18%
Syllabus Quiz	1%
FE Acknowledgement Quiz	1%
Lab Submissions	10%

You must score at least 40% on the Final Exam to receive a final grade of C or higher

Grading /2

*Tentative

Letter Grade	Percentage
A	100% - 90%
B	< 90% - 80%
C	< 80% - 70%
D	< 70% - 60%
F	< 60% - 0%

Grading /3

- Tentative and is subject to change
- Any changes will be announced both in class and on Webcourses
- Must have taken the final exam **AND** earned a score of at least 40% in the final exam to be eligible to get at least a final grade of C or higher

Grading /4

- The grade computation on Webcourses may be inaccurate

  Especially true early on the semester

- Grades are rounded to two decimal places

  For example, 89.995 → 90.00 and 89.994 → 89.99

Communication /1

- My UCF email is: **yancy.paredes@ucf.edu**

- I handle multiple sections, so please use the following subject:
    - **`Course – Section – Topic/Concern`**
    - **Example:** COP3502 – 0003 – Clarification on HW #1 Test Case

- If I have not responded within 48 hours (excluding non-working days), please see me in person (after class or at my office)

Communication /2

- Don’t use the messaging functionality on WebCourses
- For urgent or complex matters, see me during **office hours**

Communication /3

- All work should be submitted to WebCourses, **not through email**

- Constructive, actionable feedback is always welcome and appreciated

Code Submissions

- You will submit all code assignments through Gradescope

- The instructions will clearly state what is permitted and prohibited in your code

- You are responsible for the correctness of the content of your submission

Late Submission Policy

- Webcourses will automatically deduct from your grade
- A **5%** of the total grade will be deducted per hour late*
- Fractional part is rounded up, so 1 minute late is treated as 1 hour

Missing Submissions

- If your submission mysteriously disappears on Webcourses, contact **Webcourses@UCF Support**

- Include my email in the correspondence (cc)

- If the support team is unable to validate your submission, it will be presumed that you did not submit any work at all

Logistics

- Class attendance is not part of your final grade (*may be recorded*)
- Lecture *may* be recorded via Zoom (to access, login via SSO)
- Refer to Webcourses for the class and exam schedule
- Take the syllabus quiz before Friday for Financial Aid purposes

Academic Integrity

- Please **do not** cheat! =)
- Any work submitted is assumed to be written by you alone
- Do not use any AI to write any portion of your work (e.g., ChatGPT)
- Your submission may be subject to checking for similarity*

Tips to Succeed

Ask questions

Tips to Succeed /1

- Computer Programming is challenging!
  - Syntax, problem solving, math?
  - The programming language may be new to you

- Topics are cumulative in nature

- Practice, practice, practice

Tips to Succeed /2

- Try solving LeetCode problems
- Code on your own first, no AI or external help
- Attempt solving first before looking at the solution
- Don’t delay in solving problems

Tips to Succeed /3

- Don’t be content with what is provided to you
- Be curious, ask what if questions
- Try to experiment and build upon what you already know!
- Allocate a lot of time for independent practice

Tips to Succeed /4

- If you are struggling in the course, see me right away
- I would be very happy to meet with you during my office hours
- Hopefully, you don’t meet with me at the end of the term when **it is already late**

Tips to Succeed /5

- Finally, try to find patterns!

- It’s likely that the solution to a problem is something that you have already seen in the past

- Don’t simply memorize things; look at the bigger picture!

- Don’t overthink!

Skills to Learn

- Writing a code
- Tracing
- Debugging
- Critical and Logical Thinking

How to Solve Problems?

- Understand the problem
- Break down the problem into smaller pieces
- Design a solution
- Consider alternatives
- Implement the solution
- Test the solution

Homework

What is the Dunning-Kruger effect?

Can you identify what is its opposite?

Questions?